claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a3d8e54acbe21ea2a.jsonl`
- SHA-256: `8f2dbd9e797dd6dd247a5b4f359ca48de684088bae0118872df6cce257ac9f91`
- Source modified: `2026-07-02T02:54:05+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:47:11.983Z)

CONTEXT (read carefully):
- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: triage all 14 open ENGINE issues + the 1 open PR against current code. Repo: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
gh issue list --state open, then for EACH (#172 #175 #301 #326 #327 #328 #330 #331 #332 #340 #349 #384 #388 plus any others) read it (gh issue view N) and verify against current backend/ code whether still valid / partially done / fixed-but-open. Classify: quick-win (S, Claude-doable, default-safe), real-feature (M/L), owner-gated (spend/product). Pay special attention to #340 (Gemini re-bill on reprocess - a COST bug, verify the current /api/process dedup behavior at file:line) and #388 (async-job plumbing migration - relates to PR #390). Also gh pr view 390

```
--json mergeable,mergeStateStatus,title - is it conflicting/stale? Report fixed-but-open issues
as hygiene, live cost/safety issues at their true severity.
```

2. user (2026-07-02T02:47:20.287Z)

```
388     OPEN    refactor: migrate async-job plumbing
(_get_executor/_drain_due_jobs/_arm_wake_timer) out of main.py             2026-06-26T13:31:49Z
384     OPEN    ci: single-source the shared offline guard lanes + coverage floor via a
workflow_call reusable          2026-06-25T17:23:52Z
349     OPEN    WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU
~80% idle (no NVDEC/parallel decode)              2026-06-23T16:24:58Z
340     OPEN    [cost] /api/process re-runs the paid segmenter for an already-processed video ?
refresh/"Try again" re-bills Gemini     bug     2026-06-22T18:21:52Z
332     OPEN    [safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE
sending it to Gemini             2026-06-22T14:18:42Z
331     OPEN    [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase
breakdown + levers (GPU/NVENC, parallel extraction, async compile)
2026-06-22T14:14:15Z
330     OPEN    [optimization] MD5 segmentation reuse ? skip Gemini re-segmentation when the
same video (video_id) is re-uploaded               2026-06-22T14:11:32Z
328     OPEN    Surface the per-video runlog / execution trace to the user (endpoint + UI), not
just server logs                2026-06-22T13:57:58Z
327     OPEN    Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-
exempt recognized storage backends from the path-jail            2026-06-22T13:54:42Z
326     OPEN    Hosted-app observability: request tracing + replay for debugging production CUJ
failures                2026-06-22T13:54:40Z
301     OPEN    T11 follow-up: run the upload retention sweep as an automated scheduled job
(hosted alpha)      enhancement     2026-06-21T17:34:32Z
175     OPEN    Shared cloud container (OneDrive) for GPU-extractor golden data ? sync GPU box ?
dev sessions           2026-06-15T04:25:56Z
172     OPEN    Calibrate the ship-gate F1@tIoU target (currently provisional ? NOT owner-
ratified)             2026-06-14T16:40:59Z
---PR---
390     refactor(D1/P10): extract orchestration.py from main.py (god-module split, step 1)
refactor/p10-split-main OPEN    2026-06-26T14:28:06Z
```

3. user (2026-07-02T02:47:20.609Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
```

4. user (2026-07-02T02:47:31.949Z)

```
{"files":["backend/main.py","backend/orchestration.py","mypy.ini"],"mergeStateStatus":"UNKNOWN",
"mergeable":"UNKNOWN","title":"refactor(D1/P10): extract orchestration.py from main.py (god-
module split, step 1)","updatedAt":"2026-06-26T14:37:18Z"}
```

5. user (2026-07-02T02:47:34.491Z)

```
title:  [cost] /api/process re-runs the paid segmenter for an already-processed video ?
refresh/"Try again" re-bills Gemini
state:  OPEN
author: avidullu (Avi)
labels: bug
comments:       0
assignees:
projects:
milestone:
number: 340
--
**Severity: cost + correctness.** A page refresh or the SPA's "Try again" re-uploads the same
```

file and POSTs `/api/process` again. Since `video_id` is the **MD5 of the file**, the re-submit carries the **same `video_id`** ? but the engine **re-runs the full (paid) segmenter anyway** instead of returning the existing result. On the Gemini path that's a repeated, real API charge (? $0.10 per 8-min match) for zero new information, plus another multi-minute compute run.

Confirmed current behavior (`backend/main.py`)
`_execute_processing` computes `was_reingest = db_instance.get_video(video_id) is not None` ? i.e. it **detects** the video is already processed ? but uses that only for destroy-after-confirm safety (`_finalize_reingest` prunes/replaces). There is **no "already processed ? reuse, skip the segmenter" path**: it runs the segmenter and replaces the segments regardless. So a re-submit re-bills.

Live trigger (alpha run, 2026-06-22)
An 8.3-min upload finished with 27 rallies, but the SPA's 10-min poll ceiling timed out **~seconds before completion** and showed "Couldn't process ? Try again." Clicking "Try again" would have re-uploaded the identical file and re-run Gemini end-to-end. (We recovered by loading `?video=<id>` instead.) The SPA-side resume is tracked in `avidullu/khelsutra#111`; **this issue is the engine-side guardrail** so we're protected even if a re-submit slips through (defense in depth).

Proposed fix
- Make `/api/process` **idempotent by content hash**: if `video_id` already has valid `play_segments` and no explicit `force`/`reprocess` flag, **return the existing result immediately** (the queryable `video_id`) **without invoking the paid segmenter**.
- Re-processing must be **opt-in** (`force=true` / an explicit "Re-run" action) so it's never accidental.
- Optional earlier short-circuit: if the upload hashes to an existing `video_id`, skip the re-upload too.

Acceptance
- [ ] Re-submitting an already-processed video (same file/hash) returns the cached rallies with **zero** segmenter/Gemini calls (assert no Gemini call in a test).
- [ ] `force=true` still re-runs (explicit), preserving the existing re-ingest destroy-after-confirm safety.
- [ ] Default (no force) is a no-op-cost reuse.
- [ ] Pairs with `avidullu/khelsutra#111` (SPA resume so a refresh doesn't re-trigger in the first place).

Cost context: paid Gemini ? $0.10 per 8-min match; at alpha scale, accidental re-runs are real money + a nasty surprise on a metered key. The M8 cost-ledger exists but doesn't meter Gemini usage yet, so a silent re-bill wouldn't even show up.
=====341=====
title: [optimization] MD5 segmentation reuse ? skip Gemini re-segmentation when the same video (video_id) is re-uploaded
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 330
--
Ask (from the owner)
"If I upload the same video, I hope we do an MD5 checksum to reuse the Gemini segmentation."

Today
`video_id = compute_video_id(file)` **is** the MD5 of the file content, and a video's rallies are stored in `play_segments` keyed by `video_id`. But `/api/process` currently **re-runs the full segmentation** (Gemini) on every submit ? re-paying Gemini (paid) and re-waiting (~8 min on the CPU box) even for an already-segmented, byte-identical file. No reuse short-circuit found in `main.py` / `job_worker.py`.

Proposed

On `/api/process`, if `video_id` already has stored segments **for the same segmenter + relevant config**, **reuse** them and skip Gemini (return immediately with the existing rallies). Add an explicit **`force_resegment`** flag to override. The cache key already exists (MD5 `video_id`); this is mostly a short-circuit + the match condition.

Care

- Only reuse when the segmenter and segmentation-affecting config match (don't serve `gemini` segments for a `wasb_hybrid` request).
- Keep the existing "destroy-after-confirm" re-ingest semantics for an explicit re-segment.
- Big win: Gemini is paid + the CPU re-processing is the slow path; reuse makes re-opening a video instant and free.

6. user (2026-07-02T02:47:41.905Z)

```
56:    enforce_startup_checks,
95:    ``request.app.state`` ? eliminating the Optional module-level globals that forced
321:    # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
322:    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
712:    enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
934:    are present so a forced per-request override can actually dispatch + collect a result.
995:    # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded
by
1172:    was_reingest = db.get_video(video_id) is not None
1327:            was_reingest=was_reingest,
1793:def _enforce_startup_or_exit(config) -> None:
1802:        enforce_startup_checks(config, logger=logger, include_exposure=True)
1983:    _enforce_startup_or_exit(
2059:        was_reingest = db_instance.get_video(video_id) is not None
2132:                was_reingest=was_reingest,
```

7. user (2026-07-02T02:47:42.745Z)

```
Exit code 1
__pycache__
annotations
api
assets
audio
billing.py
compute
config
detectors
eval
features
flywheel.py
infrastructure
main.py
pipeline
providers
reporting
results.py
sports
stitcher
storage
tools
ui
utils
worker
  2198 backend/main.py
  2198 total
```

8. user (2026-07-02T02:47:49.871Z)

```
1140        test code that calls ``_execute_processing(req)`` still works.
1141
1142        ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
1143        Raises on unexpected internal errors ? the caller records those as a job failure
1144        """
1145        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
1146        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
1147        if config is not None and not isinstance(config, AppConfig):
1148            # Old signature: _execute_processing(req, progress=None)
1149            if req is None and db is not None and isinstance(db, ProcessRequest):
1150                req = db
1151                db = None
1152            elif req is None and isinstance(config, ProcessRequest):
1153                req = config
1154                config = None
1155        # Resolve config/db from module globals when not passed explicitly
1156        if config is None:
1157            config = global_config
1158        if db is None:
1159            db = db_instance
1160
1161        notify = progress or (lambda stage: None)
1162
1163        notify("hashing")
1164        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
1165        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
1166        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
1167        # consumers (hash / validators / segmenter) use the resolved local path.
1168        local_video = storage.acquire_local_input(
1169            req.video_path, getattr(config, "storage", None)
1170        )
1171        video_id = compute_video_id(local_video)
1172        was_reingest = db.get_video(video_id) is not None
1173
1174        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
1175        notify("validating")
1176        logger.info(f"Running validations for {req.video_path}...")
1177        val_results, val_context = registry.run_all_with_context(local_video, config)
1178        failures = [r for r in val_results if not r.passed]
1179
1180        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181        default_seg = getattr(
1182            getattr(config, "indexing", None), "default_segmenter", "motion"
1183        )
1184        seg_name = req.segmenter or default_seg
1185        segmenter_actual = None
1186        segmentation_ran = False
1187        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190        compute_actual: Optional[str] = None
1191        response: Optional[Dict[str, Any]] = None
1192        try:
1193            if failures:
```

```
1194                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1195                # status; it no longer destroys the video's prior good segments.
1196                db.add_video(
1197                    video_id,
1198                    req.video_path,
1199                    "failed",
1200                    [r.model_dump() for r in val_results],
1201                )
1202                response = {
1203                    "video_id": video_id,
1204                    "status": "failed",
1205                    "message": "Video failed validation checks.",
1206                    "results": [r.model_dump() for r in val_results],
1207                }
1208                return response
1209
1210            # 2. Run segmenter & indexer
1211            logger.info("[API] Video passed checks. Segmenting and indexing...")
1212            db.add_video(
1213                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
1214            )
1215
1216            segmenter_cls = segmenter_registry.get(seg_name)
1217            if not segmenter_cls:
1218                # Submit-time validation already 400s unknown names; this is the defensive
1219                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
```

9. user (2026-07-02T02:47:50.592Z)

```
2030        swept = db_instance.fail_stale_jobs()
2031        if swept:
2032            logger.warning(
2033                f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed."
2034            )
2035
2036        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
2037        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
2038        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
2039        _maybe_sweep_gemini_orphans(global_config)
2040
2041        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
2042        if args.video_path and not args.server and not args.app:
2043            print(f"[CLI] Processing video file: {args.video_path}")
2044            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
2045            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
2046            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
2047            storage_cfg = getattr(global_config, "storage", None)
2048            try:
2049                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
2050            except ValueError as e:
2051                print(f"[FAIL] {e}")
2052                return
2053            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
2054                print(f"[FAIL] Video file not found: {args.video_path}")
2055                return
2056            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
2057
```

```
            video_id = compute_video_id(local_video)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(
                local_video, global_config
            )
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(
                video_id, args.video_path, status, [r.model_dump() for r in val_results]
            )

            seg_name = args.segmenter or getattr(
                getattr(global_config, "indexing", None), "default_segmenter", "motion"
            )
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "=" * 40)
                print("VALIDATION SUMMARY")
                print("=" * 40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("=" * 40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(
                    video_id, local_video, max_frames=args.max_frames
                )
                segmentation_ran = True

                if isinstance(result, Failure):
                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                    print("[FAIL] Indexing halted for video.")
                    segments = []
                else:
                    segments = result.value if hasattr(result, "value") else result
                    print(
                        f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}"
                    )
                # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
                _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
            finally:
                # Always emit the per-video observability report. Never raises.
                paths = emit_indexer_report(
```

```
2121                    db=db_instance,
2122                    video_id=video_id,
2123                    # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI would
write a
2124                    # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source URI
is kept
2125                    # in DB provenance via db.add_video.
2126                    video_path=local_video,
2127                    config=global_config,
2128                    val_results=val_results,
2129                    val_context=val_context,
2130                    segmenter_requested=args.segmenter,
2131                    segmenter_actual=segmenter_actual,
2132                    was_reingest=was_reingest,
2133                    segmentation_ran=segmentation_ran,
2134                )
2135                if paths:
2136                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
2137                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
2138                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
2139            return
```

10. user (2026-07-02T02:48:00.994Z)

```
backend\main.py:287:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`,
`_get_executor`,
backend\main.py:290:# intercepts the inter-function calls (a parked job arms the PATCHED
`_arm_wake_timer`,
backend\main.py:295:    _arm_wake_timer,
backend\main.py:296:    _drain_due_jobs,
backend\main.py:297:    _get_executor,
backend\main.py:1422:    _get_executor().submit(
backend\main.py:1423:        _drain_due_jobs, request.app.state.config, request.app.state.db
backend\main.py:1635:    _get_executor().submit(
backend\main.py:1636:        _drain_due_jobs, request.app.state.config, request.app.state.db
backend\main.py:2181:        _get_executor().submit(_drain_due_jobs, global_config, db_instance)
backend\api\job_worker.py:10:`_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is
resolved THROUGH the
backend\api\job_worker.py:14:`_arm_wake_timer`, so no real daemon timer leaks in tests).
Importing `backend.main` lazily
backend\api\job_worker.py:43:def _get_executor() -> ThreadPoolExecutor:
backend\api\job_worker.py:205:def _drain_due_jobs(config: Any, db: Any) -> int:
backend\api\job_worker.py:244:    _main._arm_wake_timer(max(0.0, min(delay,
_main._MAX_DRAIN_WAKE_SEC)), config, db)
backend\api\job_worker.py:247:def _arm_wake_timer(delay_sec: float, config: Any, db: Any) ->
None:
backend\api\job_worker.py:255:        lambda:
_main._get_executor().submit(_main._drain_due_jobs, config, db),
```

11. user (2026-07-02T02:48:04.201Z)

```
title:  refactor: migrate async-job plumbing (_get_executor/_drain_due_jobs/_arm_wake_timer) out
of main.py
state:  OPEN
author: avidullu (Avi)
labels:
comments:       0
assignees:
projects:
milestone:
number: 388
--
```

Context

Follow-up to the P10 main.py split (CODE_CLEANUP_AUDIT D1). The first extraction PR moves routes, CLI, and orchestration out of `backend/main.py` but **intentionally leaves the async-job plumbing in place** because it's tightly coupled to the `db_instance`/`global_config` module globals and 8+ tests patch these symbols directly on the `backend.main` module object (e.g. `mocker.patch('backend.main._drain_due_jobs')`).

What

Move these three functions out of `backend/main.py` into a dedicated module (e.g. `backend/api/jobs.py` or `backend/orchestration/scheduler.py`):

- `_get_executor()` ? the `ThreadPoolExecutor(max_workers=1)` singleton
- `_drain_due_jobs(config, db)` ? the wake-timer-driven poll loop
- `_arm_wake_timer(seconds, config, db)` ? the self-scheduling wake timer

Why it's deferred

1. **Tight coupling to module globals**: `_drain_due_jobs` reads `db_instance`/`global_config` from the `backend.main` module object at call-time. Moving it requires either passing them explicitly or resolving via `app.state` ? a behavioral change, not a pure move.
2. **Test patches**: tests do `import backend.main as _main` then `mocker.patch('_main._drain_due_jobs')`. A move requires updating all those patch targets.
3. The P10 split is already large; this keeps PR1 focused on the mechanical extraction.

Acceptance

- [ ] The three functions move to a new module.
- [ ] `backend/main.py` no longer defines them (re-export shim removed after consumer migration).
- [ ] All tests that patch them are updated to the new patch targets.
- [ ] The `job_worker` module imports them from the new home.
- [ ] Suite green; no behavior change.

The P10 extraction PR is structured so this migration is a clean follow-up (the symbols are re-exported from `main.py` today, so the move is non-breaking when done).
=====384=====
title: ci: single-source the shared offline guard lanes + coverage floor via a workflow_call reusable
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 384
--
Follow-up from #381 review (LOW ? drift)

`nightly.yml` mirrors `ci.yml`'s offline guard lanes by **copying** them, and the coverage floor `70` is a hardcoded literal in both workflows. Nothing enforces that they stay in sync ? `ci.yml` has already recalibrated that floor once, and the next ratchet-up would silently leave the nightly on the stale value.

Proposed fix

Extract the shared **offline** guard lanes into a reusable workflow ? e.g. `.github/workflows/_offline-guards.yml` exposing them via `on: workflow_call` (with the coverage floor as an input) ? and have **both** `ci.yml` and `nightly.yml` `uses:` it. That single-sources:
- the lane set (lint · leak-scan · typecheck · import-smoke · full-suite · golden-fixtures · license-scan · build-artifact), and
- the coverage floor (one `cov-fail-under` value, passed as an input).

Why deferred from #381
The refactor has to modify `ci.yml` as well, which would break #381's single-file scope. Tracked here per the reviewer's "fine to ship as-is + file a tracked follow-up" call.

Scope / non-goals
- Offline (Tier-1) lanes only. No GPU / real-video / `run_regression.py`.
- Keep the two intentional nightly-vs-ci differences: nightly's `golden-fixtures` is single-OS (ci's `golden-crossos` covers the matrix per PR), and nightly's `notify-on-failure` job has no ci equivalent.

Refs: #381, `docs/GOLDEN_REGRESSION_FIXTURES.md`.

12. user (2026-07-02T02:48:19.255Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18
19      import logging
20      import threading
21      import traceback
22      from concurrent.futures import ThreadPoolExecutor
23      from typing import Any, Dict, Optional
24
25      logger = logging.getLogger(__name__)
26
27      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
28      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
29      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
30      # already parallelize their AI handoff internally via their own pools.
31      #
32      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
33      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
34      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
35      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
36      #   - detector timeout_sec kills hung WSL/GPU calls
37      #   - Cloud Run poll has a max_wait_sec bound
38      # A future slice should add a per-job wall-clock timeout (the executor itself
39      # cannot enforce timeouts on the Thread).
40      _job_executor: Optional[ThreadPoolExecutor] = None
41
42
43      def _get_executor() -> ThreadPoolExecutor:
44          """Lazy module-global executor; tests monkeypatch this for inline execution."""
45          global _job_executor
46          if _job_executor is None:
47              _job_executor = ThreadPoolExecutor(
48                  max_workers=1, thread_name_prefix="jobworker"
```

```
49              )
50          return _job_executor
51
52
53      class JobWaiting(Exception):
54          """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
55          WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
56          NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
57          releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
58          is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
59
60          The wiring is additive: the production segmenter path never raises this today (zero
61          behaviour change), so a job runs exactly as before. The hook is what a later slice
62          (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
63          """
64
65          def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
66              super().__init__(f"job parked for {delay_sec:.0f}s")
67              self.delay_sec = max(0.0, float(delay_sec))
68              self.progress = progress
69
70
```

13. user (2026-07-02T02:48:19.938Z)

```
280     global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
281     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
282     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
283     _instance_lock: Optional[Any] = None
284
285     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
286     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
287     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
288     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
289     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
290     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
291     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
292     from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
293         _MAX_DRAIN_WAKE_SEC,
294         JobWaiting,
295         _arm_wake_timer,
296         _drain_due_jobs,
297         _get_executor,
298         _job_to_request,
299         _maybe_record_job_cost,
300         _run_claimed_job,
301         _schedule_next_drain_if_waiting,
302     )
303
304
305     # Legacy thin wrapper for any remaining direct calls during transition.
306     # New code should import load_app_config / AppConfig from backend.config directly.
307     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
308         cfg = load_app_config(config_path)
309         return cfg.model_dump(mode="json")
```

14. user (2026-07-02T02:48:38.955Z)

{"lastCommit":"2026-06-26T14:37:15Z","mergeStateStatus":"DIRTY","mergeable":"CONFLICTING"}

15. user (2026-07-02T02:48:41.278Z)

title: [safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE sending it to Gemini
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 332
--

Ask (owner)

A **litmus test** that flags if a video is **not a racket-sports game**, so we **don't send inappropriate content (NSFW/porn) to Gemini** ? *"some users will try."* Acknowledged as non-trivial; design later.

Why it must run BEFORE Gemini

1. **ToS/abuse** ? sending explicit content to Google's API is a real violation/abuse risk.
2. **Cost** ? Gemini is paid; garbage in = wasted spend.
3. **Quality** ? non-sports input yields nonsense rallies.

Proposed design (first pass ? likely graduates to a tracked design doc)

Integration point: a new **content-safety validator** in the *existing validation phase* (the pipeline already does `Running validations?` ? `passed checks` ? segment). Reject before the gemini segmenter with a clear, friendly message (loud-failure bar). Cheap + **fully local** (no Gemini call for rejected content), on a handful of sampled frames.

Two gates:
1. **NSFW / abuse gate (the porn concern ? must-have):** a fast image classifier on ~N sampled frames ? reject if explicit. *Licensing constraint:* weights must be **MIT/Apache/BSD** (the commercial-clean guardrail) ? the model choice needs license vetting.
2. **Sports-positive gate (quality/cost filter):** "does this look like a racket-sports match?" ? **reuse the engine's existing court detection + person detection + motion**; if no court / no players / no rally-motion consistently across sampled frames ? flag as not-a-match.

Tuning / trade-offs: sample count + thresholds tunable; bias the **NSFW gate toward catching** (abuse cost is high ? a few false rejections are acceptable), bias the sports gate toward **allowing** (don't reject a real-but-unusual match). Provide a clear rejection reason in the result + (per #328) the trace.

Licensing/guardrail: no paid-LLM-as-classifier for training; any bundled weights MIT/Apache/BSD; exclude minors from any training pool (existing guardrails apply).

Relates to: the validators layer (the integration seam), #328 (surface the rejection reason/trace).
=====331=====
title: [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown + levers (GPU/NVENC, parallel extraction, async compile)
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 331
--

Measured on the live alpha (2-core CPU droplet), a real 411s (6.9 min) 1080p match ? 26 rallies

Phase	Wall time	Bound by	Lever
Upload (7 chunks)	~30 s	client upstream bandwidth	(network, not us)
Validation	~87 s	**CPU** (frame decode/validators)	GPU/NVDEC, faster CPU
Chunk extraction (6× ffmpeg re-encode + downscale)	**~5.5 min**	**CPU** (software libx264)	**GPU/NVENC**, more cores, **skip the downscale**, parallelize
Gemini upload + analysis	~1.5 min	network (upload) + Gemini API (~15s/chunk, parallel)	(fixed ? Google's side)
Reel stitch (trim+re-encode 10 clips ? concat ? watermark)	~3 min	**CPU** (per-clip re-encode)	**GPU/NVENC**; also async-compile (#329) so it doesn't 524

~65% of the wait is one thing: ffmpeg re-encoding on 2 cores (extraction + per-clip trim). The Gemini *analysis* is fast and external; the upload is client bandwidth.

Recommended levers (in impact order)

1. **GPU box (L4) with NVENC/NVDEC** ? collapses extraction + stitch from minutes to seconds; **also unlocks the owned WASB detector** (higher rally-detection quality) ? speed **and** quality. (Per ADR-012/013, GCP L4 / Cloud Run GPU.)
2. **Software (no new HW):** skip/relax the `ai_scale_width` chunk downscale (trade upload bytes for far less encode); parallelize extraction across cores; faster encode preset.
3. **Async `/api/compile`** (#329) so a slow stitch returns 202 + poll instead of a Cloudflare 524.

Baseline numbers above are a clean before/after reference once a GPU/optimizations land.

16. user (2026-07-02T02:48:52.357Z)

Found 40 files
backend\pipeline\detectors\base.py
backend\pipeline\detectors\wasb_runner.py
backend\eval\served_gate_a.py
backend\main.py
backend\config\models.py
backend\eval\rally_seg_eval.py
backend\utils\logging_setup.py
backend\storage\gdrive.py
backend\storage\ref.py
backend\utils\app_home.py
backend\pipeline\validators\relevance.py
backend\reporting\harvest.py
backend\pipeline\segmenters\motion.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\device.py
backend\pipeline\detectors\native_wasb_runner.py
backend\eval\per_user_regression.py
backend\eval\promotion.py
backend\eval\regression.py
backend\eval\segmentation_metrics.py
backend\eval\splits.py
backend\features\rally_seq.py
backend\flywheel.py
backend\infrastructure\database.py
backend\infrastructure\execution_policy.py
backend\eval\batch.py
backend\eval\eval_stats.py
backend\eval\fusion_golden.py
backend\eval\golden_fixtures.py
backend\compute\cloud_run.py
backend\config\loader.py
backend\eval\ablation.py
backend\annotations\base.py
backend\api\auth.py
backend\ui\assets\index-ukuS9wxa.js

backend\config\presets.py
backend\assets\fonts\LICENSE-OFL-1.1.txt
backend\reporting\spec.yaml
backend\assets\fonts\LICENSE-Apache-2.0.txt

17. user (2026-07-02T02:48:56.629Z)

title: Surface the per-video runlog / execution trace to the user (endpoint + UI), not just server logs
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 328
--
Feedback from the first real hosted-alpha use. After uploading a video, the owner wanted to **see the runlog and trace the execution** for *that* video ? but today the trace only lives in `journalctl -u khelsutra-engine` and the per-video `obsreport` on the box, neither user-accessible.

What the trace contains (visible in the engine log today): validation ? segmenter selected ? video duration + chunking plan ? per-chunk Gemini extraction/analysis ? rallies found ? reel compile. Rich and useful ? just not surfaced.

Proposed:
1. An endpoint to fetch a job's execution trace, e.g. `GET /api/jobs/{job_id}/trace` (or expose the per-video `obsreport` / `report.json`) ? structured steps + timings + counts.
2. A UI affordance ("view processing details" / a live progress log) on the job card so a user can watch/trace their video.
3. Correlate with the `x-correlation-id` so a UI trace ties to the server log.

Relates to #326 (request tracing/replay observability) ? this one is the **per-video pipeline** trace specifically.
=====327=====
title: Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt recognized storage backends from the path-jail
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 327
--
Today
When `security.allowed_video_root` is set (required to expose), `/api/process` runs `validate_input_path` (local-under-jail) **before** resolving the storage scheme, so `gs://bucket/x.mp4` ? `os.path.realpath` escapes the jail ? **400**. Bucket ingest is therefore impossible on the exposed alpha box. Pinned by `tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri` (gs/s3/gdrive all 400).

Want
The owner wants to ingest large videos via `gs://` on the hosted box (far more convenient than a 500 MB browser upload).

Proposed
Make `validate_input_path` / the `/api/process` preamble **scheme-aware**: either (a) allow recognized storage schemes (`gs://`, `s3://`, `gdrive-api://`) *before* the local-jail check, or

(b) apply the jail **only to local refs** (validate after `storage.resolve_input_ref`, jailing only `backend == "local"`).

? Must NOT reopen the arbitrary-local-file hole ? the jail must still fully apply to **local** paths (traversal/symlink/absolute). Add tests for both the allow (bucket URI accepted) and the reject (local traversal still 400) paths. Pairs with the `gs://` ingest the alpha CUJ + DEPLOY_ALPHA §7 currently disclaim.
=====326=====
title: Hosted-app observability: request tracing + replay for debugging production CUJ failures
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 326
--

Context

A hosted alpha upload failure was hard to debug because the failing requests **never reached the engine** ? Cloudflare Access `302`'d them at the edge, so `journalctl` showed nothing. We want first-class observability so production CUJ failures are diagnosable (and replayable) without guesswork.

What exists today

- `x-correlation-id` threaded SPA ? engine.
- Structured engine logs (`journalctl -u khelsutra-engine`) ? every request method/path/status.
- Per-video `obsreport` (observability report next to each processed video).
- **Cloudflare Zero Trust ? Logs ? Access** ? the edge allow/block/login log (where the cross-origin failure was actually visible).

Gap

- No unified **request capture/replay** store for the app's API requests.
- No easy way to **import/replay** captured requests for repro or regression.
- Edge (Access) failures aren't correlated with engine logs.

Proposed

1. Optional request-capture middleware ? a debug store (method, path, status, cid, redacted headers, timing), **default-OFF**, flag-gated.
2. A **replay harness** that loads captured requests and re-issues them against a target engine (repro + CUJ regression).
3. A debugging **runbook**: Access logs + `journalctl` + correlation-id flow.
4. Basic live-droplet monitoring (uptime / job-failure / Gemini-cost), ties to the alpha-ops hygiene item.

18. user (2026-07-02T02:49:06.690Z)

backend\main.py:83:   validate_input_path,
backend\main.py:731:   SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root`` is set ? the
backend\main.py:747:   if getattr(sec, "allowed_video_root", None):
backend\main.py:752:   "(security.allowed_video_root is set); upload to local instead."
backend\main.py:770:   validate_input_path(req.uploaded_path, allowed_root=None)
backend\main.py:782:   validate_input_path(req.source_uri, allowed_root=None)
backend\main.py:1373:   getattr(request.app.state.config, "security", None), "allowed_video_root", None
backend\main.py:1376:   validate_input_path(req.video_path, allowed_root=allowed_root)
backend\main.py:1387:   # fetches it (a fetch miss fails the job loudly). validate_input_path above still runs for every
backend\main.py:1388:   # input: its null-byte guard always applies, and a configured allowed_video_root (hosted mode)
backend\main.py:1797:   auth on without the ``allowed_video_root`` jail / CF team+AUD). A

```
strict no-op when no
backend\config\startup.py:22:rest of the posture at **boot**: the ``allowed_video_root`` path-
jail is set (else an authenticated
backend\config\startup.py:107:    # a tenant can point the engine at an ARBITRARY local file
(allowed_video_root unset = "any local
backend\config\startup.py:109:    root = getattr(sec, "allowed_video_root", None)
backend\config\startup.py:115:                "security.edge_auth_enabled is ON (engine exposed)
but security.allowed_video_root is "
backend\config\startup.py:117:                "Set allowed_video_root to a jail directory before
exposing the engine.",
backend\config\models.py:330:    allowed_video_root: Optional[str] = None
backend\config\models.py:334:    # arrive with PR-3 identity scoping. ? In hosted mode, set
allowed_video_root to
backend\utils\paths.py:58:def validate_input_path(path: str, allowed_root: Optional[str] = None)
-> None:
```

19. user (2026-07-02T02:49:07.928Z)

```
1357:    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1427:        content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1431:@app.get("/api/jobs/{job_id}/cost")
1457:@app.get("/api/jobs/{job_id}")
1596:    """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the reel
(#329).
1603:    {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
1640:        content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
```

20. user (2026-07-02T02:49:14.898Z)

```
1355    @app.post("/api/process", status_code=202)
1356    def process_video(req: ProcessRequest, request: Request):
1357        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1358
1359        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
1360        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
1361        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
1362        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
1363        """
1364        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
1365        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
1366        # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
1367        try:
1368            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1369        except edge_auth.EdgeAuthError as e:
1370            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1371
1372        allowed_root = getattr(
1373            getattr(request.app.state.config, "security", None), "allowed_video_root", None
1374        )
1375        try:
1376            validate_input_path(req.video_path, allowed_root=allowed_root)
1377            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
1378            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
1379            input_ref = storage.resolve_input_ref(
1380                req.video_path, getattr(request.app.state.config, "storage", None)
1381            )
1382        except ValueError as e:
1383            raise HTTPException(status_code=400, detail=str(e)) from e
1384        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
```

```
1385        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
1386        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
1387        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
1388        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
1389        # rejects a non-local URI as out-of-root.
1390        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1391            raise HTTPException(
1392                status_code=404, detail=f"Video file not found at '{req.video_path}'"
1393            )
1394        # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
1395        # if this box can't hold that copy. Best-effort ? an unstattable remote
(network/SDK) or an
1396        # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
1397        # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
1398        if input_ref.backend != "local":
1399            need = _remote_input_size_bytes(
1400                input_ref, getattr(request.app.state.config, "storage", None)
1401            )
1402            require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
1403        if req.segmenter and not segmenter_registry.get(req.segmenter):
1404            available = list(segmenter_registry.list_available().keys())
1405            raise HTTPException(
1406                status_code=400,
1407                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
1408            )
1409
1410        job_id = uuid.uuid4().hex
1411        if not request.app.state.db.create_job(
1412            job_id,
1413            req.video_path,
1414            req.segmenter,
1415            req.player_pool,
1416            req.max_frames,
1417            owner_id=owner_id,
1418            locale=req.locale,
1419            compute=req.compute,
1420        ):
1421            raise HTTPException(status_code=500, detail="Failed to persist job record.")
1422        _get_executor().submit(
1423            _drain_due_jobs, request.app.state.config, request.app.state.db
1424        )
1425        return JSONResponse(
1426            status_code=202,
1427            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1428        )
1429
```

21. user (2026-07-02T02:49:29.212Z)

```
title:  WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle
(no NVDEC/parallel decode)
state:  OPEN
author: avidullu (Avi)
labels:
comments:       0
assignees:
projects:
```

milestone:
number: 349
--

Summary

WASB native detection on the L4 is **decode-bound on the CPU**, not GPU-bound. On the streaming path the frame decode runs **single-threaded** (`num_workers` is forced to 0), so the L4 sits mostly idle (~20% average, bursting 0?100?0) while one CPU core does HEVC/H.264 decode via `cv2.VideoCapture`. There is **no GPU/NVDEC decode path** anywhere in the engine. Throughput on a 1080p HEVC clip was ~2× realtime (5:52 video ? ~12.5 min wall-clock).

Evidence (live, on the L4 VM)

- `nvidia-smi dmon` during a run: `sm%` bursts `0 ? 85 ? 100 ? 25 ? 0` ? the L4 does a forward pass, then waits on the next decoded batch. Average ? 20%.
- The `wasb_infer.py` process ran at ~198% CPU; GPU temp 47?77 °C, ~4.2 GB resident.
- 5:52 (352 s) 1080p HEVC, 691 MB ? started 13:50, finished 14:02 (~12.5 min).
- Installed OpenCV in the wasb env (`4.13.0`, headless PyPI wheel) reports `hasattr(cv2, "cudacodec") == False` ? no GPU decode even if code asked for it. (System `ffmpeg` *does* list `cuda` as a hwaccel, but the engine never routes decode through it.)

Root cause

1. **All decode is CPU `cv2.VideoCapture`** (OpenCV's bundled ffmpeg). No `cv2.cudacodec` / NVDEC / `-hwaccel cuda` / decord-GPU anywhere in `backend/`.
2. **Streaming forces single-process decode.** `wasb_infer.run()` sets `loader_workers = 0 if streaming else num_workers` (the in-memory frame store + the `cv2.imread` shim can't be pickled to DataLoader workers). So the fast (no-PNG) path is also the *least parallel* one.
3. The disk-frames path (`stream_video=false`) **can** use `num_workers>1` to parallelise decode/read and keep the GPU fed, but pays a full PNG extraction first (slow + disk). The 2026-06-20 deploy report already noted streaming "starves the GPU" on long clips and recommended the disk path for them.

Options (in rough order of effort)

- **Config-only mitigation (testable now, a trade-off, not a fix):** `indexing.wasb.stream_video=false` + `indexing.wasb.num_workers>1` ? parallel CPU decode keeps the L4 fed. Costs a full PNG extraction + disk. (Side benefit: also dodges the streaming resume-cache bug ? see companion issue.)
- **Batched/threaded streaming decode:** prefetch + decode frames on a small thread pool feeding the existing in-memory shim, so streaming isn't single-threaded. Keeps the no-PNG win without the disk cost.
- **True GPU decode (NVDEC):** decode on the L4 via `cv2.cudacodec.VideoReader` (needs a CUDA-built OpenCV), `PyNvVideoCodec`/`decord`-GPU, or an `ffmpeg -hwaccel cuda` pipe. Biggest win on long clips; biggest change (new dep + tensor-on-GPU handoff, must preserve the byte-identical frame contract the pipeline relies on).

Impact / severity

High for cost/throughput on the GPU-VM profile: you pay ~$0.70?0.85/hr for an L4 that's ~80% idle during detection because decode is the bottleneck. Fixing decode parallelism (or NVDEC) is the main lever to cut per-match wall-clock and make the L4 worth its cost.

Environment: GCP L4 VM (`g2-standard-4`), wasb env torch 1.11.0+cu113, OpenCV 4.13.0 (no cudacodec), `stream_video=true`, observed 2026-06-23.

=====301=====
title: T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted alpha)
state: OPEN
author: avidullu (Avi)
labels: enhancement
comments: 0

assignees:
projects:
milestone:
number: 301
--

Context

The T11 **upload retention sweep** landed in #299 (merged, `cb7394b`): `tools/cleanup_caches.py` was extended to purge ? past a retention window ? uploaded source videos and orphaned `.partial-<id>` files in `security.upload_dir` (default `output/uploads`), alongside the existing regenerable-cache cleanup. It is **dry-run by default**, keeps reels/DB/CSVs durable, and emits a machine-readable plan via `--json`.

That PR delivered the **tool**. What it did **not** do is run it **unattended**: today an operator must manually run `python -m tools.cleanup_caches --apply` on the box. Before/while non-owner (alpha) users upload footage, the retention window should be enforced automatically without a human in the loop.

Goal

Wire the existing sweep into an **automated scheduled job** so stale uploads are purged on the owner-set retention window (uploads **7d**; reels kept durable) without manual intervention, with loud structured logging captured to observability.

Do not re-implement the sweep ? schedule the existing tool. The `--json` output (`{windows, uploads_dir, reclaimable_bytes, purge[], by_category[]}`) is already shaped for a wrapper/log sink.

What's needed

- A scheduler appropriate to the deployment:
  - **Single-box alpha** (engine on one host): a host **cron** or **systemd-timer** entry that runs the sweep daily.
  - **Cloud Run serving path**: a **Cloud Scheduler ? job** invocation (or a sidecar), consistent with `COMPUTE_DECOUPLED_SERVING/`.
- The scheduled invocation runs `--apply` with `--uploads-retention-days 7` and `--uploads-dir` matching `security.upload_dir` (consider resolving from config rather than hardcoding the path).
- **Default-OFF / opt-in** so the single-user local install is never surprised by an unattended deletion job.
- **Observability**: capture the `--json` plan + the per-item `PURGED/FAILED` lines to logs each run (counts + bytes freed + the active windows), per the launch quality bar (loud, structured, correlation id).
- **Document** it in the deploy runbook (engine `deploy/` and/or `khelsutra/deploy/DEPLOY_ALPHA.md`).

Design considerations / open questions

- **In-flight safety:** the retention window already protects recent `.partial-<id>` files (an in-flight chunked upload < window is never swept). Confirm the cadence + window leave a comfortable margin over the max upload duration.
- **Active-job safety:** a *finished* upload may still be referenced by an in-progress `/api/process` job. Decide whether the scheduled apply should additionally skip uploads referenced by a non-terminal row in the `jobs` table (defense-in-depth beyond the time window). The window alone is probably sufficient for 7d, but worth a deliberate call.
- **Per-user subdirs (PR-3):** the sweep currently leaves an unknown upload **subdir** untouched (`classify` returns `upload subdir` = keep). When identity scoping lands, the job/tool will need to recurse per-user.
- **Apply guard:** since the scheduled job runs `--apply`, gate it behind an explicit config flag + keep the conservative window so a misconfiguration can't nuke fresh uploads.

Acceptance criteria

- [ ] A scheduled mechanism runs `tools/cleanup_caches.py` on a cadence (e.g. daily) in the

hosted/alpha deployment.
- [ ] It applies the owner-set window (uploads 7d, reels durable) and writes the structured `--json` plan + outcome to logs/observability.
- [ ] Default-OFF / opt-in; the local single-user path is unaffected.
- [ ] In-flight + active-job safety is reasoned about and documented (window margin, optional jobs-table check).
- [ ] The deploy runbook documents how to enable + verify it.
- [ ] A test/smoke confirms the wrapper invokes the tool with the right flags (the tool's own logic is already unit-tested).

References

- PR #299 (the tool) · `tools/cleanup_caches.py` · `tests/test_cleanup_caches.py`
- `backend/api/uploads.py` (where uploads land) · `backend/config/models.py` `SecurityConfig.upload_dir`
- Cache-hygiene policy (retention table) · `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` §9 (T11, khelsutra repo) · the `ui-driven-cloud-serving-gap` thread

Filed as the scheduling follow-up to #299; the sweep tool itself is done and merged.
=====175=====
title: Shared cloud container (OneDrive) for GPU-extractor golden data ? sync GPU box ? dev sessions
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 175
--
Problem. The rally-quality measurement is split across machines by capability:
- `backend/eval/fusion_golden.py` (shuttle+person feature **extraction**) needs the videos, trajectory CSVs, `.rallies.csv` golden labels, and a GPU/WSL stack ? **GPU box only**.
- `backend/eval/fusion_compare.py` / `backend/eval/ablation.py` (the LOVO **litmus** ? ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the small `*_golden_features.json` ? **any session** (incl. a CPU dev container).

Today these artifacts live in gitignored `output/` per-machine + scattered local folders (`...\Annotation Setup\Collect\Trajectories\`, `...\Golden Labelled\`). Nothing syncs them, so a CPU dev/agent session can't see the GPU box's extraction.

Plan. Use a OneDrive shared folder as the canonical exchange: `C:\Users\avidu\OneDrive\rally-annotator\golden-data\` (present + signed-in on both machines under the same account ? auto-syncs). GPU box writes `*_golden_features.json` (+ trajectory CSVs, manifests) there; CPU sessions read from it. Raw videos stay local (tens of GB). Full design + workflow + guardrails: **`docs/GOLDEN_DATA_SHARING.md`**.

Tasks:
- [x] Stand up `OneDrive\rally-annotator\golden-data\{features,trajectories,manifests,baselines}\` + a README (this session).
- [ ] GPU box: run `fusion_golden --out-dir <shared>/features/<tag>` (or extract local then copy the JSONs); confirm it syncs to the dev machine.
- [ ] CPU session: `fusion_compare --features-dir <shared>/features/<tag> --record` + `ablation --granularity group` ? real person-feature ?F1; light up the `skipif` golden-litmus tests.
- [ ] (optional ergonomics) a `RALLY_GOLDEN_DIR` env var honored by `fusion_golden`/`fusion_compare`/`ablation`; a tiny `scripts/sync_golden.py` copier.
- [ ] Keep the folder **private** (Proprietary corpus) and **outside the repo** (never committed).

Refs: `docs/GOLDEN_DATA_SHARING.md`, the NEXT_STEPS "Next-session pickup" anchor, the P3 fusion litmus.
=====172=====
title: Calibrate the ship-gate F1@tIoU target (currently provisional ? NOT owner-ratified)

```
state:  OPEN
author: avidullu (Avi)
labels:
comments:       0
assignees:
projects:
milestone:
number: 172
```
--
Context. `training/gen0/harness.py::gate_verdict` references ship-gate F1@tIoU figures sourced from the merged plan (`docs/POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md`, M0):

- Floor (necessary): heuristic LOVO ~**0.480**
- Multi-court reference ("worth serving"): **0.66**
- Proposed for high-P/R + significance: **F1@tIoU0.5 ? 0.70** on multi-court folds, paired-sign p<0.05 vs heuristic

These numbers are PROVISIONAL and have NOT been ratified by the owner. They were listed in the plan as references/proposals derived from a tiny corpus; the owner has explicitly stated they did not commit to that figure. PR #170 should not be read as owner ratification of the gate target.

Ask. Calibrate and ratify the ship-gate target properly once the golden corpus grows to a defensible multi-court size:
- Pick the floor and the "worth serving" threshold from real multi-court LOVO data (not the n=2 synthetic / tiny-corpus numbers).
- Decide the significance bar (paired sign test / bootstrap CI) at that corpus size.
- Until then, `gate_verdict` should treat the F1 target as **uncalibrated/provisional** (a blocker that keeps `ship=False` on F1 grounds), not as "ratified".

Refs: PR #170 (clears the old hard `UNDEFINED` block), plan M0 quality table, #169 (merged plan). C3 (WASB provenance) remains the separate commercial blocker.

22. user (2026-07-02T02:49:29.220Z)

```
40          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
41              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
42          return name
43
44
45      def resolve_under_root(path: str, root: str) -> str:
46          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
47          rp = os.path.realpath(path)
48          rr = os.path.realpath(root)
49          try:
50              contained = os.path.commonpath([rp, rr]) == rr
51          except ValueError:  # different drives on Windows
52              contained = False
53          if not contained:
54              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
55          return rp
56
57
58      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
59          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
60
61          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
62          so the local 'point it at any of my videos' workflow is preserved. Setting an
63          allowed_root (a future hosted/tenant deployment) enforces containment.
64          """
65          if "\x00" in path:
66              raise ValueError("video_path must not contain a null byte")
67          if allowed_root:
```

```
68              resolve_under_root(path, allowed_root)
69
```

23. user (2026-07-02T02:49:42.865Z)

```
tools/cleanup_caches.py
---deploy tree---
deploy/:
cloudrun
digitalocean
gcp

deploy/gcp/:
README.md
buckets.env
create_gpu_vm.sh
nightly_regression
ops-agent-startup.sh
provision.sh
teardown.sh
```

24. user (2026-07-02T02:49:44.597Z)

```
backend\pipeline\detectors\wasb_infer.py:274:    ``num_workers=0``.
``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
backend\pipeline\detectors\wasb_infer.py:410:    num_workers: int = 4,
backend\pipeline\detectors\wasb_infer.py:431:      ``cvtColor(bgr, BGR2RGB)``, verified byte-
identical). Streaming forces ``num_workers=0``
backend\pipeline\detectors\wasb_infer.py:551:        loader_workers = 0 if streaming else
num_workers
backend\pipeline\detectors\wasb_infer.py:553:            ds, batch_size=batch_size,
shuffle=False, num_workers=loader_workers
backend\pipeline\detectors\wasb_infer.py:567:            f"(total {windows_total},
batch={batch_size}, workers={loader_workers}, "
backend\pipeline\detectors\wasb_infer.py:767:            num_workers=0,
backend\pipeline\detectors\wasb_infer.py:860:        num_workers=args.num_workers,
```

25. user (2026-07-02T02:49:45.922Z)

```
162:def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
203:        "ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue #172"
234:    verdict = gate_verdict(eval_res, floor)
```

26. user (2026-07-02T02:50:02.892Z)

```
24:GEMINI_USD = (
25:    "gemini_usd"  # provider-reported cost, ALREADY in USD (a passthrough, not rate-×)
28:# Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a USD
amount).
39:    layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
51:    gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
52:    if gemini:
53:        breakdown[GEMINI_USD] = gemini
54:    total += gemini
```

27. user (2026-07-02T02:50:04.298Z)

```
---capture middleware---
backend/eval/fusion_golden.py
backend/eval/golden_fixtures.py
backend/eval/golden_real_fixtures.py
backend/pipeline/detectors/stub_runner.py
backend/pipeline/detectors/wasb_infer.py
```

28. user (2026-07-02T02:50:13.678Z)

```
backend/pipeline/validators/scene_cut.py
---golden data sharing doc---
GOLDEN_REGRESSION_FIXTURES.md
---onedrive---
onedrive dir missing/inaccessible
```

29. user (2026-07-02T02:50:26.010Z)

```
---ui asset---
1
```

30. user (2026-07-02T02:50:29.025Z)

Summary

First step of the **P10 / D1** item from `docs/CODE_CLEANUP_AUDIT_2026-06-24.md`: split the `backend/main.py` god-module. This PR extracts the **processing + compile orchestration** into a new `backend/orchestration.py`, reducing `main.py` from **1797 ? 1217 lines** (32% reduction).

> **Scope note:** This is the first of a multi-step split (P10a). The CLI (`main()`) and route handlers stay in `main.py` for now ? they're tightly coupled to the module globals (`global_config`/`db_instance`) and the `app` object, so moving them needs careful global-mutation handling (tracked in #388 / P10b). This PR proves the extraction pattern and delivers the biggest single win.

What moved ? `backend/orchestration.py` (635 lines)

Function	Purpose
`_finalize_reingest`	Destroy-AFTER-confirm segment commit
`_ingest_remote_segments`	Cloud job result ? DB rows
`_cloud_available`	Can this box dispatch a cloud job?
`_maybe_dispatch_remote`	Cloud Run dispatch + ingest (default-OFF)
`_execute_processing`	Full hash ? validate ? segment ? report pipeline
`_CompileError`	Submit-time-validatable compile problem
`_resolve_compile`	Shared submit/worker compile resolution
`_execute_compile`	Worker handler for a compile job

Proof this is behavior-preserving

AST-verified (not just "looks safe"):
- ? All 8 extracted functions are **AST-identical** to the originals in `main.py` (except 2 intentional, semantically-neutral changes to avoid a circular import ? see below).
- ? All **30 retained functions** in `main.py` (routes + CLI + helpers) are **byte-identical to master**.
- ? The **21 consumers** that do `import backend.main as m` and access `_execute_processing` / `_resolve_compile` / etc. keep working unchanged ? the symbols are re-exported via the import.

The 2 intentional AST deltas (circular-import avoidance)

1. `_maybe_dispatch_remote`: the `ProcessRequest` annotation is now a string forward-ref (`"ProcessRequest"`) instead of a bare `Name` ? avoids importing `backend.main` at module load.
2. `_execute_processing`: the backward-compat shim that reads `db_instance`/`global_config` now imports them **lazily** from `backend.main` inside the function body (was a bare module-global reference) ? defers the circular import to call-time.

Both are annotation/import-mechanics only; runtime behavior is identical.

Follow-ups (tracked)

- **#388** ? migrate the async-job plumbing (`_get_executor`/`_drain_due_jobs`/`_arm_wake_timer`) out of `main.py`.
- **P10b** ? extract the CLI (`main()`) and route handlers, handling the `global` statement +

```
`app` coupling.
===comments===
```

31. user (2026-07-02T02:50:43.482Z)

```
5409fc4 refactor(api): extract job-worker drain/wake loop from main into backend.api.job_worker
(#234)
---main.py history since 06-26---
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
```

32. user (2026-07-02T02:50:54.584Z)

```
2026-06-18 16:17:25 +0530 refactor(api): extract job-worker drain/wake loop from main into
backend.api.job_worker (#234)
---
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)
(#339)
918a4ff feat(jobs): worker self-scheduling drain loop honoring WAIT_AND_RESUME (EXECUTION_POLICY
P3a) (#149)
3c18cb9 feat(api): async job model (#16) - 202+job_id, jobs table, worker, crash sweep (#87)
```

33. user (2026-07-02T02:51:14.971Z)

```
---uploads sched---
```

34. user (2026-07-02T02:51:32.633Z)

```
{"closedAt":null,"state":"OPEN","title":"Resilient processing UX: survive page refresh +
continually-updating progress"}
---dup check 330 vs 340---
{"state":"CLOSED","title":"[reliability] /api/compile is synchronous ? Cloudflare 524 on slow
stitches; make it async (202 + poll) like /api/process"}
```

35. user (2026-07-02T02:51:41.810Z)

```
.github/workflows/ci.yml:145:        run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
.github/workflows/nightly.yml:139:        run: python run_all_tests.py --cov=backend
--cov=training --cov-report=term --cov-fail-under=70
ci.yml
nightly.yml
```

36. user (2026-07-02T02:52:15.162Z)

```
1       from typing import Any
2
3       import cv2
4       import numpy as np
5
6       from backend.pipeline.validators.base import ValidationResult, VideoValidator
7
8
9       class RelevanceValidator(VideoValidator):
10          @property
11          def name(self) -> str:
12              return "relevance_check"
```

```
13
14          @property
15          def description(self) -> str:
16              return "Checks if the video content is relevant to badminton by scanning for
court color palettes and geometric features."
17
18          def validate(
19              self, video_path: str, config: Any, context: dict[str, Any]
20          ) -> ValidationResult:
21              cap = cv2.VideoCapture(video_path)
22              if not cap.isOpened():
23                  return ValidationResult(
24                      validator_name=self.name,
25                      passed=False,
26                      message="Failed to open video file for relevance validation.",
27                  )
28
29              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
30              if total_frames <= 0:
31                  cap.release()
32                  return ValidationResult(
33                      validator_name=self.name, passed=False, message="Invalid frame count."
34                  )
35
36              # Sample 10 frames across the video
37              num_samples = 10
38              step = max(1, total_frames // num_samples)
39
40              green_ratios = []
41
42              # Get sport profile config
43              sport_name = config.sport
44              from backend.sports import sport_registry
45
46              try:
47                  sport_profile = sport_registry.get(sport_name)
48                  sport_rel_cfg = sport_profile.get_relevance_config()
49              except KeyError:
50                  sport_rel_cfg = {}
51
52              # Typed config: validation.relevance provides Pydantic defaults;
53              # sport profile overrides only when the user hasn't explicitly set values.
54              rel_cfg = config.validation.relevance
55
56              # Read bounds: user config wins over sport profile, with Pydantic defaults as
fallback.
57              hsv_lower = np.array(
58                  rel_cfg.court_green_lower
59                  or sport_rel_cfg.get("court_green_lower")
60                  or [35, 40, 40]
```

37. user (2026-07-02T02:52:51.585Z)

```
tests\test_api_endpoints.py:809:    Counterweight =
test_process_hosted_mode_rejects_nonlocal_uri (gs:// -> 400 when allowed_video_root
tests\test_api_endpoints.py:903:def test_process_hosted_mode_rejects_nonlocal_uri(env):
```

38. user (2026-07-02T02:54:04.963Z)

```
Structured output provided successfully
```